

WSL上で「Pythonから呼んでいるC++」をデバッグする流れ

目的はこれです。

Pythonを実行する

↓

simulation_manager が C++ を呼ぶ

↓

C++側の関数で止める

↓

C++の変数を見る

0. 前提理解

Python debugger では C++ の中には入れません。

Python debugger → Pythonコードを見る

C++ debugger → C++コードを見る

だから今回は、**C++ debugger** で **Python** を起動する形にします。

gdb / VSCode C++ debugger

↓

Python起動

↓

PythonがC++を呼ぶ

↓

C++のbreakpointで止まる

1. WSLにgdbがあるか確認

WSLのターミナルで:

```
gdb --version
```

表示が出ればOK。

なければ:

```
sudo apt update
```

```
sudo apt install gdb build-essential
```

2. C++側をDebugビルドにする

C++を普通にビルドすると、最適化されて変数が見えにくくなります。

デバッグ用にはこれが必要です。

-g : デバッグ情報を入れる

-O0 : 最適化を切る

cpp_build.sh を使っている場合

普段これでビルドしているなら:

```
bash cpp_build.sh
```

cpp_build.sh の中を見る。

```
cat cpp_build.sh
```

探す文字列:

```
g++
clang++
cmake
make
setup.py
pip
-O3
-O2
-Ofast
-DNDEBUG
```

直接 g++ / clang++ している場合

例えば中身がこうなら:

```
g++ -O3 -shared ...
```

デバッグ時はこうします。

```
g++ -g -O0 -shared ...
```

-O3, -O2, -Ofast は一旦外す。

CMakeを使っている場合

例えば:

```
mkdir build
cd build
cmake ..
make -j
```

なら、Debugにする。

```
rm -rf build
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Debug ..
make -j
```

ここで最後の `..` は、1つ上のディレクトリです。

`.` = 今いる場所
`..` = 1つ上

例えば:

```
project/
  CMakeLists.txt
  build/
```

このとき `build/` の中で:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

と打つと、1つ上の `project/` にある `CMakeLists.txt` を読みに行きます。

3. Pythonが読む `.so` が更新されたか確認

Pythonから呼ばれるC++は、たいてい `.so` という共有ライブラリになっています。

確認:

```
find . -name "*.so" -ls
```

ここで表示される `.so` の更新日時が新しくなっているか見る。

重要なのはこれです。

C++を書き換えた

↓

再ビルドした

↓

Pythonが読む .so が更新された

ここがズれていると、breakpointが効きません。

4. まずC++が本当に呼ばれているか確認

C++側の入口っぽい関数に、一時的にこれを入れる。

```
#include <iostream>
```

そして関数の中に:

```
std::cout << "C++ called" << std::endl;
```

例えば:

```
void Simulation::step(...) {  
    std::cout << "C++ step called" << std::endl;  
  
    // 元の処理  
}
```

その後、再ビルド。

```
bash cpp_build.sh
```

Pythonを普通に実行して、表示が出るか見る。

```
python your_script.py
```

出れば:

PythonからこのC++関数は本当に呼ばれている

出なければ:

違う関数を見ている

Pythonが違う .so を読んでいる

そもそもの処理ルートを通っていない

5. VSCodeをWSLモードで開く

WSLのプロジェクトフォルダで:

`code .`

VSCode左下にこう出ていればOK。

WSL: Ubuntu

出ていなければ、Windows側ではなくWSL側で開けていない可能性があります。

6. VSCodeにC++拡張を入れる

VSCodeの拡張機能で、WSL側に以下を入れる。

C/C++ extension

Microsoftのやつです。

Pythonも使うなら:

Python extension

も入れておく。

7. `.vscode/launch.json` を作る

プロジェクト直下に `.vscode/launch.json` を作ります。

```
project/  
  .vscode/  
    launch.json  
simulation_manager/  
  cpp_build.sh  
  your_script.py
```

中身の例:

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {
```

```
"name": "C++ Debug Python",
"type": "cppdbg",
"request": "launch",
"program": "${workspaceFolder}/.venv/bin/python",
"args": ["your_script.py"],
"cwd": "${workspaceFolder}",
"MIMode": "gdb",
"stopAtEntry": false,
"setupCommands": [
  {
    "description": "Enable pretty-printing",
    "text": "-enable-pretty-printing",
    "ignoreFailures": true
  }
]
}
```

仮想環境を使っていない場合

ここを:

```
"program": "${workspaceFolder}/.venv/bin/python"
```

こうする。

```
"program": "/usr/bin/python3"
```

普段こう実行している場合

例えば普段:

```
python train.py --config configs/debug.yaml
```

なら **args** はこう。

```
"args": ["train.py", "--config", "configs/debug.yaml"]
```

つまり、コマンドを分解するとこうです。

```
python train.py --config configs/debug.yaml
```

対応:

```
"program": "${workspaceFolder}/.venv/bin/python",
```

```
"args": ["train.py", "--config", "configs/debug.yaml"]
```

8. C++ファイルにbreakpointを置く

C++ファイルをVSCodeで開きます。

例えば:

```
Simulation::step(...)
Simulation::reset(...)
simulate(...)
calc(...)
```

など、Pythonから呼ばれていそうな関数の行番号の左側をクリック。

赤丸がつきます。

赤丸 = breakpoint

9. VSCodeでデバッグ実行

VSCode左側の「実行とデバッグ」を開く。

設定から:

C++ Debug Python

を選ぶ。

実行ボタンを押す。

流れはこう。

VSCode C++ debugger 起動

↓

Python実行

↓

PythonがC++を呼ぶ

↓

C++のbreakpointに到達

↓

そこで止まる

10. 止まったら何が見えるか

止まると、VSCode左側にこういう欄が出ます。

VARIABLES
WATCH
CALL STACK
BREAKPOINTS

見るのは主にこれ。

VARIABLES

例えば:

Locals
state = ...
action = ...
reward = ...
done = ...
this = ...

みたいにC++側の変数が見えます。

11. 操作ボタン

上に出るデバッグ操作はこうです。

Continue : 次のbreakpointまで進む
Step Over : 次の行へ進む。関数の中には入らない
Step Into : 関数の中に入る
Step Out : 今の関数から出る
Restart : 再実行
Stop : 停止

Python debugger とかなり似ています。

12. Watchで見たい変数を固定表示する

左側の **WATCH** に変数名を書くと、その値を常に見られます。

例:

state


```
action
reward
this->xxx
```

C++のメンバ変数なら:

```
this->state_
this->reward_
```

みたいにかけることがあります。

13. よくある詰まり

breakpointで止まらない

原因候補:

- C++がDebugビルドされていない
- Pythonが古い .so を読んでいる
- breakpointを置いた関数が呼ばれていない
- launch.jsonのPythonが普段使っているPythonと違う
- .soがまだ読み込まれていない

対処:

```
find . -name "*.so" -ls
```

で **.so** の更新日時を見る。

C++入口に:

```
std::cout << "C++ called" << std::endl;
```

を入れて、本当にそこを通るか確認する。

変数が **<optimized out>** になる

原因はだいたい最適化です。

```
-O2
-O3
-Ofast
```

が残っている可能性が高いです。

対処:

```
-g -O0
```

でビルド。

CMakeなら:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

赤丸が灰色になる

共有ライブラリ `.so` がまだ読み込まれていないだけのことがあります。

PythonがC++モジュールをimportするまでは、breakpointが未解決になる場合があります。

実行してC++が読み込まれると有効になることがあります。

Pythonのimportで古いものを読んでいる

これもかなり多いです。

確認用にPythonで:

```
import your_cpp_module
print(your_cpp_module.__file__)
```

これで、実際に読んでいる `.so` の場所が出ます。

その場所が、自分がビルドした `.so` と一致しているか確認します。

14. 最短ルート

まずはこれだけやればいいです。

```
gdb --version
```

次に `cpp_build.sh` をDebug用にする。

```
-g -O0 を付ける
-O3 / -O2 / -Ofast を外す
```

ビルド。

```
bash cpp_build.sh
```

.so 更新確認。

```
find . -name "*.so" -ls
```

C++入口にログ。

```
std::cout << "C++ called" << std::endl;
```

再ビルドしてPython実行。

```
python your_script.py
```

表示が出たら、VSCodeで `launch.json` を作ってC++ breakpointを置く。

15. 全体像

- ① gdb確認
- ↓
- ② C++をDebugビルド
- ↓
- ③ .so更新確認
- ↓
- ④ C++関数が本当に呼ばれるかログ確認
- ↓
- ⑤ VSCodeをWSLモードで開く
- ↓
- ⑥ launch.json作成
- ↓
- ⑦ C++にbreakpoint
- ↓
- ⑧ C++ debuggerでPythonを起動
- ↓
- ⑨ breakpointで停止
- ↓
- ⑩ VARIABLESでC++変数を見る

これが一連の流れです。

うん。ここは初歩から分解した方がいいです。要点はこれです。

まず何が起きているか

君のプログラムは、たぶんこういう二層構造です。

Pythonコード

↓ 呼び出す

C++で書かれた高速な計算部分

Python debugger が見られるのは基本ここまで。

Pythonの変数 

Pythonの関数 

C++を呼ぶ直前 

C++内部の変数 

C++内部を見るには、**C++**用のデバッガが必要です。

WSLならそれがほしい **gdb** です。

デバッガの種類が違う

Python debugger

Pythonコードを見る道具。

```
x = 1
y = 2
z = cpp_simulation.step(x, y)
```

ここで Python debugger は、

```
x = 1
y = 2
```

までは見られます。

でも `cpp_simulation.step()` の中身が C++ なら、そこには入れません。

C++ debugger

C++コードを見る道具。

```
int step(int x, int y) {
    int z = x + y;
    return z;
}
```

C++ debugger なら、

```
x = 1  
y = 2  
z = 3
```

みたいに見られます。

君がやりたいこと

やりたいことはこれです。

Pythonを実行する

↓

途中でC++が呼ばれる

↓

C++の中で止める

↓

C++の変数を見る

そのために、

Pythonを Python debugger で起動する

のではなく、

Pythonを C++ debugger で起動する

という形にします。

ここが一番大事です。

例えると

Python debugger は、

Python世界の監視カメラ

C++ debugger は、

C++世界の監視カメラ

今は Python世界の監視カメラだけで C++工場の中を見ようとしている状態です。
だから見えません。

最初に確認すべきこと

まず WSL のターミナルでこれを打ってください。

```
gdb --version
```

出ればOK。

出なければ入れます。

```
sudo apt update
```

```
sudo apt install gdb build-essential
```

次に大事なこと: C++をデバッグ用にビルドする

C++は普通にビルドすると、速くするために中身が見づらくなります。

だからデバッグ用にします。

必要なのは主にこれ。

-g : 変数名や行番号を残す

-O0 : 最適化を切る

CMakeを使っているなら:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

```
make -j
```

手動コンパイルなら:

```
g++ -g -O0 ...
```

VSCodeでやる場合の考え方

VSCodeにはいくつかのデバッガがあります。

Python debugger

C++ debugger

今回は **C++ debugger** を使います。

ただし起動するプログラムは C++ではなく、Pythonです。

つまりこう。

```
"program": "/home/君のユーザー名/プロジェクト/.venv/bin/python",
```

```
"args": ["実行したいPythonファイル.py"]
```

意味は、

C++デバッガで Python を起動する

PythonがC++を呼ぶ

C++のbreakpointで止まる

です。

最小の launch.json

`.vscode/launch.json` にこう書きます。

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Python calling C++",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/.venv/bin/python",
      "args": ["your_script.py"],
      "cwd": "${workspaceFolder}",
      "MIMode": "gdb",
      "stopAtEntry": false
    }
  ]
}
```

`your_script.py` は、普段実行している Python ファイル名に変えます。

例えば普段こう実行しているなら：

```
python train.py
```

こうします。

```
"args": ["train.py"]
```

どうやって止めるか

C++ファイルを開いて、止めたい行の左側をクリックします。

赤い点がつきます。

これが breakpoint です。

```
int Simulation::step(...) {
  // ← ここに赤い点を置く
}
```

その状態で VSCode のデバッグ実行を押す。

Pythonが走って、C++のその行に来たら止まります。

止まったら VSCode 左側に変数が出ます。

Locals
state = ...
action = ...
reward = ...
done = ...

ただし、最初はここで詰まりやすい

breakpoint が効かない

原因候補:

C++が Debug ビルドされていない
Pythonが別の .so を読んでいる
breakpoint を置いた関数が実際には呼ばれていない
C++ライブラリがまだ読み込まれていない

特に多いのはこれ。

編集しているC++と、実際にPythonが使っているC++ライブラリが違う

つまり、

C++を書き換えた
でも再ビルドしていない
Pythonは古い .so を読んでいる

という状態です。

超実用的な第一歩

いきなりデバuggに入る前に、C++側の入口にこれを入れるのがいいです。

```
std::cout << "C++ step called" << std::endl;
```

Pythonを実行してこれが出れば、

このC++関数は本当に呼ばれている

と確認できます。

これが出ないなら、breakpoint以前の問題です。

まとめ

君が今理解すべき最小単位はこれです。

Python debuggerではC++内部は見えない
C++内部を見るにはgdbが必要
gdbでPythonを起動する
PythonがC++を呼んだ瞬間にC++側で止める
C++はDebugビルドしておく必要がある

まずは次の2つだけ確認すればいいです。

```
gdb --version
```

と、

普段Pythonをどういうコマンドで実行しているか

そこが分かれば、`launch.json` を君の環境用に具体化できます。

ビルドする時、`bash cpp_build.sh`みたいなのでやってるけど、shファイルの中とかもいじる必要ある？

はい、いじる必要がある可能性が高いです。

見るべきは `cpp_build.sh` の中で、C++をコンパイルしている箇所です。

まず探すべき文字列

`cpp_build.sh` の中に、こういうのがあるはずです。

```
g++  
clang++  
cmake  
make  
setup.py  
pip install
```

直接 `g++` / `clang++` している場合

こういう感じなら：

```
g++ -O3 -shared ...
```

デバッグ用にはこう変えます。

```
g++ -g -O0 -shared ...
```

ポイントはこれ。

-g ：デバッグ情報を入れる
-O0 ：最適化を切る

-O3 や -O2 があれば、デバッグ時は外します。

CMakeを使っている場合

こういう行があるなら:

```
cmake ..  
make -j
```

こうします。

```
cmake -DCMAKE_BUILD_TYPE=Debug ..  
make -j
```

または、既存の build ディレクトリがあるなら一度消す方が安全です。

```
rm -rf build  
mkdir build  
cd build  
cmake -DCMAKE_BUILD_TYPE=Debug ..  
make -j
```

Python拡張としてビルドしている場合

こういう行なら:

```
python setup.py build_ext --inplace
```

`setup.py` 側で `extra_compile_args` を見ます。

例えば:

```
extra_compile_args=["-O3"]
```

これをデバッグ時は:

```
extra_compile_args=["-g", "-O0"]
```

にします。

よくある最悪パターン

`cpp_build.sh` にこういうのがある場合:

```
-O3  
-Ofast  
-DNDEBUG
```

これはデバッグにはかなり不向きです。

-O3 / -Ofast : 最適化で変数が見えなくなる
-DNDEBUG : assertなどが無効になる

デバッグ時は基本こう。

-g -O0

必要なら:

-UNDEBUG

おすすめは **debug / release** を分けること

cpp_build.sh をこういう形にすると扱いやすいです。

```
#!/usr/bin/env bash
set -e

BUILD_TYPE=${1:-Release}

if [ "$BUILD_TYPE" = "Debug" ]; then
    CXXFLAGS="-g -O0"
else
    CXXFLAGS="-O3"
fi

echo "Build type: $BUILD_TYPE"
echo "CXXFLAGS: $CXXFLAGS"

# 例
g++ $CXXFLAGS -shared ...
```

実行は:

```
bash cpp_build.sh Debug
```

普段の高速版は:

```
bash cpp_build.sh Release
```

結論

cpp_build.sh は確認・修正対象です。

特に見るべきはこの3つ。

1. -O3 / -O2 / -Ofast が入っていないか
2. -g が入っているか
3. CMakeなら -DCMAKE_BUILD_TYPE=Debug になっているか

まず **cpp_build.sh** の中身を貼ってくれば、どこをどう変えるべきかそのまま具体的に指摘できます。

その..**みたいなやつ**って何入れるの？

..**は「1つ上のディレクトリ」**という意味です。

CMake の例でいうと:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

この最後の `..` は、

「1つ上の階層にある `CMakeLists.txt` を見に行け」

という意味です。

典型形

プロジェクトがこうなっている場合：

```
my_project/  
  CMakeLists.txt  
  src/  
  build/
```

`build/` の中に移動してから：

```
cd build  
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

と打ちます。

このとき `..` は `my_project/` を指します。

```
現在地: my_project/build  
.. : my_project
```

つまり

`..` に何か文字を入れるのではなく、そのまま `..` と書くことが多いです。

ただし、今いる場所によって変わります。

例えば `CMakeLists.txt` が今いる場所にあるなら：

```
cmake -DCMAKE_BUILD_TYPE=Debug .
```

になります。

```
. = 今いるディレクトリ  
.. = 1つ上のディレクトリ
```

まずはこれで確認できます。

```
find . -name CMakeLists.txt
```

出てきた場所を見れば、`cmake` の最後に `.` を入れるべきか、`..` を入れるべきか分かります。

その後何すんだっけ

次はこの順番です。かなり機械的に進めればOKです。🔥

1. Debugビルドする

CMakeなら例:

```
rm -rf build
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Debug ..
make -j
```

または普段の方式がこれなら:

```
bash cpp_build.sh
```

`cpp_build.sh` の中で `-g -O0` が付くようにする。

2. Pythonが読む `.so` を更新する

C++をビルドしたあと、Pythonから呼ばれる `.so` / 拡張モジュールが更新されている必要があります。

確認:

```
find . -name "*.so" -ls
```

日付が新しくなっていればOK。

3. VSCodeをWSLモードで開く

WSLのプロジェクトフォルダで:

```
code .
```

VSCode左下がこうなっていればOKです。

WSL: Ubuntu

4. `.vscode/launch.json` を作る

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "C++ Debug Python",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/.venv/bin/python",
      "args": ["普段実行しているPythonファイル.py"],
      "cwd": "${workspaceFolder}",
    }
  ]
}
```

```
"MIMode": "gdb",
"stopAtEntry": false
}
]
```

仮想環境がないなら:

```
"program": "/usr/bin/python3"
```

5. C++側にブレークポイントを置く

C++の入口っぽいところに赤丸を置く。

例:

```
step(...)
reset(...)
simulate(...)
calc(...)
```

最初は「Pythonから必ず呼ばれる関数」に置くのがいいです。

6. VSCodeでデバッグ実行

左の「実行とデバッグ」から:

C++ Debug Python

を選んで実行。

うまくいくと:

Python開始

↓

C++関数呼び出し

↓

C++の赤丸で停止

↓

Variables にC++変数が出る

まず最低限やること

今やるべきはこれです。

```
gdb --version
```

次に:

```
bash cpp_build.sh
```

その後:

```
find . -name "*.so" -ls
```

`.so` が更新されているか確認。

そこまで行ったら、VSCode側の `launch.json` に進めばいいです。

6\ VSCodeでデバッグ実行 ----- 左の「実行とデバッグ」から: C++ Debug Python を選んで実行。うまくいくと: Python開始 ↓ C++関数呼び出し ↓ C++の赤丸で停止 ↓ Variables にC++変数が出る

これってなんかすんの? 表示出てくるの?

はい。表示が出ます。ただし「自動で全部出る」というより、ブレークポイントで止まった瞬間に **VSCode** の画面がデバッグ表示に切り替わる感じです。

起きること

VSCodeで実行すると、まずPythonが普通に走ります。

Python実行開始

C++の赤丸を置いた行に到達すると、そこで止まります。

C++の行で一時停止

するとVSCode左側にこういう欄が出ます。

VARIABLES
WATCH
CALL STACK
BREAKPOINTS

特に見るのは **VARIABLES** です。

そこに例えば:

```
state = ...  
action = ...  
reward = ...  
this = ...
```

みたいに、C++側の変数が出ます。

画面イメージ

左側:

VARIABLES
Local
state: 3
action: 1
reward: 0.0

中央:

C++コード

止まっている行が黄色っぽく表示される

上:

▶ 続行

↶ ステップオーバー

↓ ステップイン

何进行操作するのか

止まったら、上のボタンで進めます。

Continue : 次のブレークポイントまで進む

Step Over : 次の行へ進む

Step Into : 関数の中に入る

Step Out : 今の関数から出る

Python debugger とかなり似ています。

何も出ない場合

多くはこのどれかです。

C++のブレークポイントまで到達していない

C++がDebugビルドされていない

Pythonが古い.soを読んでいる

launch.jsonのPython実行ファイルが違う

まずはC++の入口にこれを入れて、

```
std::cout << "C++ called" << std::endl;
```

Python実行時に表示されるか確認するとよいです。

一連の流れを詳細にまとめて？

WSL上で「Pythonから呼んでいるC++」をデバッグする流れ

目的はこれです。

Pythonを実行する

↓

simulation_manager が C++ を呼ぶ

↓

C++側の関数で止める

↓

C++の変数を見る

0. 前提理解

Python debugger では C++ の中には入れません。

Python debugger → Pythonコードを見る
C++ debugger → C++コードを見る

だから今回は、**C++ debugger** で **Python** を起動する形にします。

gdb / VSCode C++ debugger

↓

Python起動

↓

PythonがC++を呼ぶ

↓

C++のbreakpointで止まる

1. WSLにgdbがあるか確認

WSLのターミナルで:

```
gdb --version
```

表示が出ればOK。

なければ:

```
sudo apt update
```

```
sudo apt install gdb build-essential
```

2. C++側をDebugビルドにする

C++を普通にビルドすると、最適化されて変数が見えにくくなります。

デバッグ用にはこれが必要です。

-g : デバッグ情報を入れる

-O0 : 最適化を切る

cpp_build.sh を使っている場合

普段これでビルドしているなら:

```
bash cpp_build.sh
```

cpp_build.sh の中を見る。

```
cat cpp_build.sh
```

探す文字列:

g++

clang++

```
cmake
make
setup.py
pip
-O3
-O2
-Ofast
-DNDEBUG
```

直接 **g++** / **clang++** している場合

例えば中身がこうなら:

```
g++ -O3 -shared ...
```

デバッグ時はこうします。

```
g++ -g -O0 -shared ...
```

-O3, **-O2**, **-Ofast** は一旦外す。

CMakeを使っている場合

例えば:

```
mkdir build
cd build
cmake ..
make -j
```

なら、Debugにする。

```
rm -rf build
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Debug ..
make -j
```

ここで最後の **..** は、1つ上のディレクトリです。

. = 今いる場所

.. = 1つ上

例えば:

```
project/
  CMakeLists.txt
  build/
```

このとき **build/** の中で:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

と打つと、1つ上の `project/` にある `CMakeLists.txt` を読みに行きます。

3. Pythonが読む `.so` が更新されたか確認

Pythonから呼ばれるC++は、たいてい `.so` という共有ライブラリになっています。

確認:

```
find . -name "*.so" -ls
```

ここで表示される `.so` の更新日時が新しくなっているか見る。

重要なのはこれです。

C++を書き換えた

↓

再ビルドした

↓

Pythonが読む `.so` が更新された

ここがズれていると、breakpointが効きません。

4. まずC++が本当に呼ばれているか確認

C++側の入口っぽい関数に、一時的にこれを入れる。

```
#include <iostream>
```

そして関数の中に:

```
std::cout << "C++ called" << std::endl;
```

例えば:

```
void Simulation::step(...) {  
    std::cout << "C++ step called" << std::endl;
```

```
    // 元の処理  
}
```

その後、再ビルド。

```
bash cpp_build.sh
```

Pythonを普通に実行して、表示が出るか見る。

```
python your_script.py
```

出れば:

PythonからこのC++関数は本当に呼ばれている

出なければ:

違う関数を見ている

Pythonが違う .so を読んでいる

そもそもその処理ルートを通っていない

5. VSCodeをWSLモードで開く

WSLのプロジェクトフォルダで:

code .

VSCode左下にこう出ていればOK。

WSL: Ubuntu

出ていなければ、Windows側ではなくWSL側で開けていない可能性があります。

6. VSCodeにC++拡張を入れる

VSCodeの拡張機能で、WSL側に以下を入れる。

C/C++ extension

Microsoftのやつです。

Pythonも使うなら:

Python extension

も入れておく。

7. `.vscode/launch.json` を作る

プロジェクト直下に `.vscode/launch.json` を作ります。

```
project/  
  .vscode/  
    launch.json  
simulation_manager/  
  cpp_build.sh  
your_script.py
```

中身の例:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "C++ Debug Python",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/.venv/bin/python",
      "args": ["your_script.py"],
      "cwd": "${workspaceFolder}",
      "MIMode": "gdb",
      "stopAtEntry": false,
      "setupCommands": [
        {
          "description": "Enable pretty-printing",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

仮想環境を使っていない場合

ここを:

```
"program": "${workspaceFolder}/.venv/bin/python"
```

こうする。

```
"program": "/usr/bin/python3"
```

普段こう実行している場合

例えば普段:

```
python train.py --config configs/debug.yaml
```

なら **args** はこう。

```
"args": ["train.py", "--config", "configs/debug.yaml"]
```

つまり、コマンドを分解するということです。

```
python train.py --config configs/debug.yaml
```

対応:

```
"program": "${workspaceFolder}/.venv/bin/python",
```

```
"args": ["train.py", "--config", "configs/debug.yaml"]
```

8. C++ファイルにbreakpointを置く

C++ファイルをVSCodeで開きます。

例えば:

```
Simulation::step(...)
Simulation::reset(...)
simulate(...)
calc(...)
```

など、Pythonから呼ばれていそうな関数の行番号の左側をクリック。

赤丸がつきます。

赤丸 = breakpoint

9. VSCodeでデバッグ実行

VSCode左側の「実行とデバッグ」を開く。

設定から:

C++ Debug Python

を選ぶ。

実行ボタンを押す。

流れはこう。

VSCode C++ debugger 起動

↓

Python実行

↓

PythonがC++を呼ぶ

↓

C++のbreakpointに到達

↓

そこで止まる

10. 止まったら何が見えるか

止まると、VSCode左側にこういう欄が出ます。

VARIABLES

WATCH
CALL STACK
BREAKPOINTS

見るのは主にこれ。

VARIABLES

例えば:

Locals
state = ...
action = ...
reward = ...
done = ...
this = ...

みたいにC++側の変数が見えます。

11. 操作ボタン

上に出るデバッグ操作はこうです。

Continue : 次のbreakpointまで進む
Step Over : 次の行へ進む。関数の中には入らない
Step Into : 関数の中に入る
Step Out : 今の関数から出る
Restart : 再実行
Stop : 停止

Python debugger とかなり似ています。

12. Watchで見たい変数を固定表示する

左側の **WATCH** に変数名を書くと、その値を常に見られます。

例:

state
action
reward
this->xxx

C++のメンバ変数なら:

this->state_
this->reward_

みたいに書けることがあります。

13. よくある詰まり

breakpointで止まらない

原因候補:

- C++がDebugビルドされていない
- Pythonが古い .so を読んでいる
- breakpointを置いた関数が呼ばれていない
- launch.jsonのPythonが普段使っているPythonと違う
- .soがまだ読み込まれていない

対処:

```
find . -name "*.so" -ls
```

で **.so** の更新日時を見る。

C++入口に:

```
std::cout << "C++ called" << std::endl;
```

を入れて、本当にそこを通るか確認する。

変数が **<optimized out>** になる

原因はだいたい最適化です。

- O2
- O3
- Ofast

が残っている可能性が高いです。

対処:

```
-g -O0
```

でビルド。

CMakeなら:

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

赤丸が灰色になる

共有ライブラリ **.so** がまだ読み込まれていないだけのことがあります。

PythonがC++モジュールをimportするまでは、breakpointが未解決になる場合があります。

実行してC++が読み込まれると有効になることがあります。

Pythonのimportで古いものを読んでいる

これもかなり多いです。

確認用にPythonで：

```
import your_cpp_module
print(your_cpp_module.__file__)
```

これで、実際に読んでいる `.so` の場所が出ます。

その場所が、自分がビルドした `.so` と一致しているか確認します。

14. 最短ルート

まずはこれだけやればいいです。

```
gdb --version
```

次に `cpp_build.sh` をDebug用にします。

```
-g -O0 を付ける
-O3 / -O2 / -Ofast を外す
```

ビルド。

```
bash cpp_build.sh
```

`.so` 更新確認。

```
find . -name "*.so" -ls
```

C++入口にログ。

```
std::cout << "C++ called" << std::endl;
```

再ビルドしてPython実行。

```
python your_script.py
```

表示が出たら、VSCodeで `launch.json` を作ってC++ breakpointを置く。

15. 全体像

- ① gdb確認
- ↓
- ② C++をDebugビルド
- ↓
- ③ .so更新確認
- ↓
- ④ C++関数が本当に呼ばれるかログ確認
- ↓
- ⑤ VSCodeをWSLモードで開く
- ↓
- ⑥ launch.json作成
- ↓
- ⑦ C++にbreakpoint
- ↓
- ⑧ C++ debuggerでPythonを起動
- ↓
- ⑨ breakpointで停止
- ↓
- ⑩ VARIABLESでC++変数を見る

これが一連の流れです。